Testing - Design Decisions Document

1. Hybrid Architecture (Mocked Context and Live Database Interaction)

Our testing suite avoids pure unit testing or heavy integration testing in favor of a hybrid setup. We mock the Jakarta REST container layer (ContainerRequestContext) using Mockito to fake our login contexts, but we let the service layer talk directly to a live database.

- **Design Implication:** This lets us quickly check that our endpoint logic and SQL queries actually work with real database constraints, without the headache of spinning up a massive application server or an embedded database engine.

2. Live Remote Database Connection via Dotenv

Instead of using a simple in-memory database (like H2), our test suite targets the live PostgreSQL instance hosted on the university server (bronto.ewi.utwente.nl). We pull the database credentials securely out of a .env file during the @BeforeClass setup.

- **Design Implication:** By doing this, we make sure our tests run against the exact same Postgres dialect used in production, saving us from unexpected bugs. The downside is that our tests completely rely on the campus network stability and need an isolated schema to run smoothly.

3. Lifecycle-Driven Database Cleanup (Before and After Routines)

To keep the database clean while running tests, every single test class runs a raw JDBC cleanUp() or cleanUsers() routine both *before* and *after* each test case execution.

- **Design Implication:** By explicitly executing DELETE FROM Person WHERE user_id = ? using standard JDBC prepared statements, we guarantee that each test starts with a blank slate. This stops leftover data from ruining other tests and prevents random rows from cluttering the database if an assertion fails.

4. Deterministic Test Identifiers (Static Values over Auto-Increment)

We deliberately chose to use fixed, static constants for our user IDs (like TEST_USER_ID = 9999, USER_1 = 9998, USER_2 = 9999) instead of writing code to generate random IDs or relying on database auto-incrementing.

- **Design Implication:** This makes our test conditions highly predictable and easy to reproduce when debugging. It also makes our cleanup scripts straightforward, though it means we can't run tests at the exact same time on the same database schema without breaking things.

5. Defensive Testing Strategy (Negative Path and Edge Case Prioritization)

Instead of just testing standard "happy paths" where everything goes right, most of our resource test coverage specifically targets edge cases and failure paths (like checking for BAD_REQUEST on null contexts, NOT_FOUND on missing items, or weak password patterns).

- **Design Implication:** This builds a defensive codebase. It proves that our REST endpoints fail safely and return the correct HTTP status codes right away, blocking bad data before it ever hits our deeper business logic.